

Distributed SpMV Multiplication

Luca Agostinelli

Abstract—Sparse matrix-vector multiplication (SpMV) is distributed across multiple GPUs with GPU-aware MPI, extending a prior single-GPU study to the multi-process setting. A row-cyclic baseline with selective ghost-value exchange is compared against an NCCL transport, a 2D checkerboard partitioning, and alternative partitioning strategies (RCM, Block). On ten large SuiteSparse matrices, row-cyclic strong scaling degrades with P on most matrices regardless of transport, exposing a PCIe-interconnect-bound implementation that the alternatives reverse on most matrices at no added communication cost.

I. INTRODUCTION AND BACKGROUND

Sparse Matrix-Vector Multiplication (SpMV), $y = Ax$, is a memory-bandwidth-bound kernel central to iterative solvers, graph analytics, and scientific computing [1]. A companion paper benchmarked SpMV on a *single* NVIDIA A30 GPU [2], showing throughput is capped by one device’s memory capacity and compute budget. This work addresses that limit by distributing SpMV across *multiple* GPUs using GPU-aware MPI, avoiding host staging. This work reuses the COO/CSR representations and local GPU kernels of [2] as the per-rank compute stage; the contribution here is entirely in how A is partitioned across processes and how needed-but-unowned x values are obtained. Unlike the shared-memory, single-device setting of [2], this work runs as independent processes (one per GPU, SPMD-style) exchanging data via CUDA-aware MPI, which accepts device pointers directly as communication buffers and moves data GPU-to-GPU without host staging.

As an alternative transport for the ghost exchange, this work also evaluates NVIDIA’s Collective Communications Library (NCCL), which implements topology-aware collective and point-to-point primitives for GPU-to-GPU traffic. We implement a row-cyclic baseline with GPU-aware ghost exchange, and explore two alternative directions (an NCCL transport and a 2D checkerboard partitioning that caps the number of communication peers) plus two alternative partitioning strategies evaluated against the row-cyclic baseline. Overall, this work delivers a correctness-validated, communication-aware distributed SpMV, evaluated through strong- and weak-scaling studies and load-balance and communication-volume analysis.

II. DISTRIBUTED SPMV DESIGN

Every evaluated scheme follows the same generic pipeline, differing only in three independent choices: the row-ownership function, the communication pattern for data a rank needs but does not own, and the local SpMV kernel (cuSPARSE or ACC, both from [2]).

Every configuration shares the same driver skeleton. At start-up, rank 0 partitions the matrix once: it assigns each row (and, for the 2D scheme, each column) an owning rank,

reads the full matrix, buckets nonzeros by owner, and ships each shard via `Scatter/Scatterv`. Every SpMV call then repeats two steps: an exchange step obtains whatever x data (or, for the 2D scheme, partial y contributions) the local kernel needs but the rank does not own, and a kernel step runs the unchanged local SpMV kernel of [2], identical across every configuration. The 2D scheme alone adds a final reduction step that sums partial y contributions into the owned result.

Not every driver uses every piece: the naive full-broadcast reference has no ghost bookkeeping (its exchange step is just a broadcast of x); the 2D scheme has no ghost concept at all but adds the final reduction.

A. Partitioning strategies

1) *Row-Cyclic (1D)*: Uses $owner(i) = i \bmod P$, where $P = |processes|$: row i belongs to rank $i \bmod P$, at local index $\lfloor i/P \rfloor$, both computable independently by every rank with no coordination. This scatters adjacent rows across ranks by construction, needing no knowledge of the matrix’s structure. Every rank thus holds two index spaces: global, used only during initial distribution, and compact local, used by every subsequent SpMV call.

2) *Checkerboard (2D)*: This scheme arranges ranks into a $P_r \times P_c$ process grid ($P_r =$ largest divisor of P , $P_r \leq \sqrt{P}$; $P_c = P/P_r$) and assigns ownership jointly by row and column: $owner(i, j) = (i \bmod P_r, j \bmod P_c)$. Column ownership no longer follows row ownership, so every rank’s set of possible row-owners (P_c of them) and column-owners (P_r of them) is fixed and known at partition time, giving a fixed communication topology capped at $\sqrt{P}$ peers.

3) *RCM and Block*: Two alternative ownership functions, both reusing the row-cyclic ghost-exchange machinery generalized to an arbitrary $owner[]/local[]$ array in place of the closed-form modulo. Block splits rows into P equal contiguous ranges in the matrix’s original order. RCM instead reorders the matrix by Reverse Cuthill–McKee (a breadth-first traversal visiting neighbors in ascending-degree order) before splitting it into NNZ-balanced blocks, requiring rank 0 to build a full symmetrized adjacency structure, an $O(nnz)$ preprocessing cost.

B. Communication strategies

1) *Full Broadcast (1D)*: The naive reference broadcasts the entire x vector to every rank on every call (`MPI_Bcast`), regardless of how much a rank’s local rows actually reference.

2) *Ghost/Halo Exchange (1D)*: Under 1D row partitioning, a rank’s local rows can reference columns owned by any other rank. At start-up, each rank rewrites its local column indices into a compact local space (owned columns first, then ghost

columns grouped by owning peer) and determines, via one `Alltoall` on ghost counts, which of its own entries each peer needs in return. Every SpMV call then exchanges only these ghost lists: point-to-point (`Isend/Irecv`) with MPI, or grouped `ncclSend/ncclRecv` in the NCCL variant.

3) *Broadcast/Reduce (2D)*: Each rank broadcasts its owned x segment along its column communicator (a fixed group of P_r column-owners), computes a local *partial* y against its row-and-column block, then sums partial contributions along its row communicator (`MPI_Reduce/ncclReduce`) to obtain the final value for its owned rows.

C. Distributed Matrix Reading

The common driver performs PARTITION on rank 0 alone: one process reads the Matrix Market file, buckets nonzeros by owner, and ships each shard via `Scatter/Scatterv`. An opt-in alternative instead has every rank read and parse only its own byte range, routing entries to their owner via `Alltoallv` so no rank holds the whole matrix. Validated correct on all matrices, it is 1.96–3.75 $\times$ faster than the centralized baseline at $P=2$ and 3.55–7.38 $\times$ faster at $P=4$ across all matrices tested; see the repository [3] for full results.

III. EXPERIMENTAL SETUP

A. Hardware and Software Environment

All experiments run on a single node with four PCIe-only NVIDIA A30 GPUs (Ampere, no NVLink), one MPI rank per GPU, for $P \in \{1, 2, 3, 4\}$. Each GPU negotiates PCIe Gen4 $x16$, a ~ 31.5 GB/s unidirectional ceiling roughly 30 $\times$ below its own on-device memory bandwidth (933 GB/s), so every communication cost reported here is PCIe-bound.

B. Dataset

An initial evaluation on the matrix set of [2] showed strong scaling degrading almost uniformly regardless of transport or partitioning, leaving little to compare; this evaluation therefore uses ten substantially larger SuiteSparse matrices instead (up to 1.02 billion nonzeros, Table I).

C. Synthetic Matrices for Weak Scaling

Weak scaling holds work per rank fixed as P grows, so each P uses its own synthetic Erdős–Rényi matrix, generated once and shared across drivers, with nonzeros/rank fixed at ≈ 1.5 million. A *primary* family ($\approx 75\,000$ rows/rank, 20 nnz/row) saturates the ghost fraction (the fraction of a rank’s remote index range its ghost list references) above 99% at every P ; a *secondary* family (750 000 rows/rank, 2 nnz/row) instead keeps it partial, from 63% at $P=2$ down to 39% at $P=4$.

D. Metrics and Correctness

Timing. Every driver times communication and local computation jointly, as a single repeated unit (10 untimed warm-up calls followed by 30 timed repetitions). Reported $T(P)$ is the wall-clock bottleneck (the maximum total time across ranks).

Speedup and throughput. Speedup is $T(1)/T(P)$ and effective throughput is $2 \cdot \text{nnz}_{\text{total}}/T(P)$ GFLOP/s ($T(P)$ = wall-clock bottleneck).

TABLE I: Dataset: large-scale SuiteSparse matrices. $\bar{d}$ = avg. NNZ/row.

Matrix	Rows	NNZ	$\bar{d}$	Domain
mycielskian19	393 215	903 194 710	2 297.0	Synthetic
Queen_4147	4 147 110	329 499 284	79.5	FEM
com-Orkut	3 072 441	234 370 166	76.3	Social network
arabic-2005	22 744 080	639 999 458	28.1	Web graph
nlpkt240	27 993 600	774 472 352	27.7	KKT optimization
uk-2005	39 459 925	936 364 282	23.7	Web graph
webbase-2001	118 142 155	1 019 903 190	8.6	Web graph
kmer_V1r	214 005 017	465 410 904	2.2	Genome graph
europe_osm	50 912 018	108 109 320	2.1	Road network
mawi_201512020330	226 196 185	480 047 894	2.1	Traffic graph

TABLE II: Strong-scaling speedup $T(1)/T(4)$, ACC kernel.

Matrix	bcast	ghost/MPI	ghost/NCCL	2D/NCCL
Queen_4147	0.43 $\times$	0.22 $\times$	0.87 $\times$	1.74$\times$
arabic-2005	0.25 $\times$	0.15 $\times$	0.75 $\times$	0.98 $\times$
com-Orkut	0.69 $\times$	0.52 $\times$	1.25$\times$	2.08$\times$
europe_osm	0.15 $\times$	0.03 $\times$	0.16 $\times$	0.28 $\times$
kmer_V1r	0.16 $\times$	0.04 $\times$	0.23 $\times$	0.29 $\times$
mawi_201512020330	0.15 $\times$	0.18 $\times$	0.12 $\times$	0.21 $\times$
mycielskian19	2.27$\times$	2.31$\times$	2.85$\times$	2.76$\times$
nlpkt240	0.25 $\times$	0.10 $\times$	0.86 $\times$	0.96 $\times$
uk-2005	0.23 $\times$	0.12 $\times$	0.73 $\times$	0.88 $\times$
webbase-2001	0.18 $\times$	0.06 $\times$	0.39 $\times$	0.46 $\times$

Load balance and communication volume. Per-rank NNZ counts give a load-balance ratio $\max/\text{avg}$ across ranks; communication volume follows directly from each driver’s own tracked element counts, converted to bytes (32-bit floats).

Correctness. As in [2], every result is validated element-wise against a serial CPU reference ($\varepsilon_{\text{abs}} = \varepsilon_{\text{rel}} = 10^{-2}$).

IV. RESULTS AND DISCUSSION

All raw results are available in the public repository [3].

A. Strong Scaling

Table II reports $S(4) = T(1)/T(4)$ for all four strategies side by side. No strategy reverses the sign of strong scaling on most of the matrix set: $S(4) < 1$ for every configuration on 7 of 10 matrices. The dense, compute-bound `mycielskian19` wins under every strategy; `com-Orkut` and `Queen_4147` join it only under NCCL/2D, not the naive or 1D/MPI baselines. Excluding `mycielskian19`, 1D/MPI’s $S(4)$ ranges 0.03–0.52 (`europe_osm-com-Orkut`) and still loses to naive broadcast on most matrices despite moving far less data. 2D checkerboard raises $S(4)$ further on 9 of 10 matrices (`Queen_4147` 1.74 $\times$, `com-Orkut` 2.08 $\times$). Throughput peaks on `mycielskian19`: 429 GFLOP/s at $P=4$ under 1D ghost/NCCL, up from 189 at $P=1$ (the only matrix where both throughput and wall-clock time improve monotonically). These results are consistent with prior multi-GPU SpMV work, where communication overhead has similarly been found to outweigh compute savings as GPU count grows [4].

B. Weak Scaling

Figure 1 shows weak-scaling efficiency $T(1)/T(P)$ for four partitioning strategies (1D ghost, 2D checkerboard, RCM, Block; NCCL, cuSPARSE) on both synthetic families. No strategy comes close to ideal scaling in either regime. Under

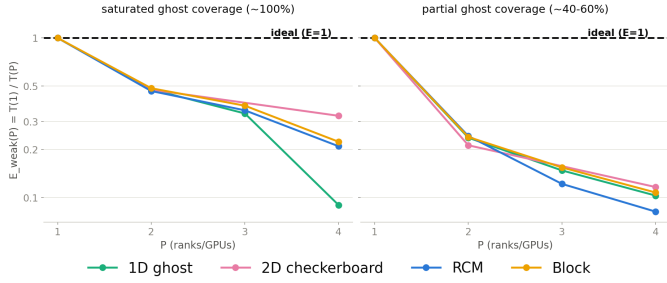


Fig. 1: Weak-scaling efficiency, saturated vs. partial ghost coverage.

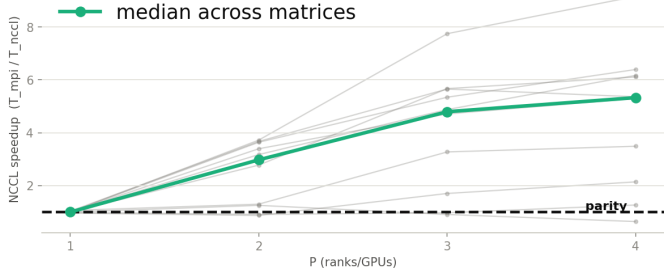


Fig. 2: NCCL vs. MPI point-to-point speedup for 1D ghost exchange across P . Thin lines are individual matrices, thick line is the median.

saturated ghost coverage ($\sim 100\%$, left panel), 2D checkerboard is the clear winner at $P=4$ ($E_{\text{weak}} \approx 0.33$, its capped peer count pays off even with no locality to exploit), RCM and Block trail close behind (≈ 0.21 – 0.23), and 1D ghost is worst (≈ 0.10). Under partial coverage (~ 40 – 60% , right panel), all four strategies converge to nearly the same low efficiency (≈ 0.09 – 0.13 at $P=4$): once 1D ghost already has some selectivity of its own to exploit, the advantage of capping peers or preserving locality largely disappears.

C. MPI Point-to-Point vs. NCCL Transport

Figure 2 isolates the transport choice alone (1D ghost exchange, identical partitioning and kernel) across P . NCCL wins on 8 of 10 matrices already at $P=2$ (median $3.0\times$), and on 9 of 10 by $P=4$ (median $5.3\times$, up to $9.2\times$ on `nlpkkt240`).

D. Row-Cyclic vs. Checkerboard Partitioning

Table III compares the 2D checkerboard layout against 1D row-cyclic partitioning, both over NCCL. At $P=1$, 2D loses on 8/10 matrices: the degenerate 1×1 grid still pays a full Bcast+Reduce round trip every call, while 1D’s ghost exchange is free when there is nothing to exchange. The picture flips quickly as P grows: 2D already wins on 6/10 matrices at $P=2$, and on 9/10 by $P=4$ (`mycielskian19` is the sole, narrow loss at $0.97\times$), as the round-trip’s fixed cost becomes negligible next to 1D’s poorly-selective exchange volume.

E. Load Balance and Communication Volume

Table IV reports per-rank NNZ imbalance ($\max/\text{avg}$) for both schemes and 1D ghost exchange’s communication volume relative to naive broadcast, all at $P=4$. Row-cyclic

TABLE III: 2D vs. 1D speedup (both NCCL, ACC). > 1 mean 2D is faster.

Matrix	$P=1$	$P=2$	$P=4$
Queen_4147	$1.04\times$	$1.06\times$	$2.07\times$
arabic-2005	$0.97\times$	$0.88\times$	$1.27\times$
com-Orkut	$1.11\times$	$1.03\times$	$1.85\times$
europe_osm	$0.90\times$	$1.48\times$	$1.57\times$
kmer_V1r	$0.87\times$	$1.19\times$	$1.07\times$
mawi_201512020330	$0.83\times$	$0.67\times$	$1.45\times$
mycielskian19	$1.00\times$	$0.98\times$	$0.97\times$
nlpkkt240	$0.97\times$	$0.88\times$	$1.07\times$
uk-2005	$0.97\times$	$1.17\times$	$1.16\times$
webbase-2001	$0.92\times$	$1.15\times$	$1.08\times$

TABLE IV: Per-rank NNZ imbalance ($\max/\text{avg}$, 1D and 2D) and 1D ghost-exchange communication volume relative to naive broadcast ($P = 4$).

Matrix	1D imbalance	2D imbalance	ghost/bcast vol.
Queen_4147	$1.00\times$	$1.00\times$	75.0%
arabic-2005	$1.00\times$	$1.00\times$	42.2%
com-Orkut	$1.00\times$	$1.00\times$	71.7%
europe_osm	$1.00\times$	$1.81\times$	47.6%
kmer_V1r	$1.00\times$	$1.00\times$	34.7%
mawi_201512020330	$2.32\times$	$1.88\times$	19.0%
mycielskian19	$1.00\times$	$1.00\times$	75.0%
nlpkkt240	$1.00\times$	$1.35\times$	62.1%
uk-2005	$1.00\times$	$1.00\times$	42.6%
webbase-2001	$1.00\times$	$1.01\times$	36.9%

is near-perfectly balanced on 9 of 10 matrices ($\leq 1.00\times$); `mawi_201512020330` is the outlier ($2.32\times$). 2D checkerboard is instead less balanced on `europe_osm` ($1.81\times$) and `nlpkkt240` ($1.35\times$) (joint $i \bmod P_r, j \bmod P_c$ ownership lets row/column density correlate into specific blocks, unlike row-cyclic’s averaging) yet more balanced than 1D on `mawi_201512020330` itself ($1.88\times$ vs. $2.32\times$).

Communication volume shows the same matrix-dependence as strong scaling: 1D ghost recovers 19% (`mawi_201512020330`) to 75% (`Queen_4147/mycielskian19`) of naive broadcast’s per-rank bytes at $P=4$. 2D checkerboard, by construction, lands at almost exactly $1\times$ naive’s volume at $P=4$ (a true 2×2 grid) and exactly $2\times$ at $P=1$, on every matrix.

F. Structured vs. Unstructured Matrices

Figure 3 contrasts two FEM matrices (`Queen_4147`, `nlpkkt240`) against two power-law graphs (`webbase-2001`, `com-Orkut`) at $P=4$. Structure does not predict the outcome: `Queen_4147` saturates ghost volume to 75% and wins only $2.56\times$; `nlpkkt240`, also FEM but only 62% saturated, wins $7.64\times$. The power-law pair diverges the same way: `com-Orkut` saturates to 72% and wins just $1.80\times$, while `webbase-2001`’s locality-preserving ordering keeps saturation at 37% and wins $8.22\times$, the largest margin of the four. Ghost-fraction saturation, not the structured/unstructured label, is what predicts whether selective exchange pays off.

G. Other Partitioning Alternatives

Transport engineering changes a strong-scaling curve’s *constant factor*, not its *direction*: even under 2D/NCCL, 7 of 10 matrices still degrade from $P=1$ to $P=4$. This section instead

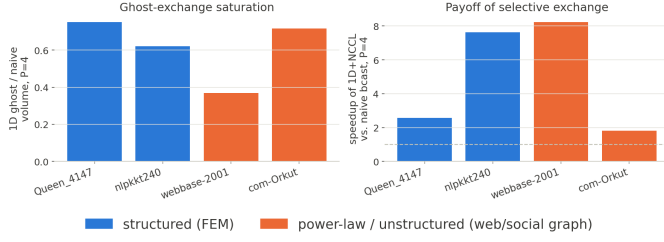


Fig. 3: Ghost-exchange saturation and 1D/NCCL speedup over naive broadcast, $P=4$: structured FEM matrices vs. power-law graphs.

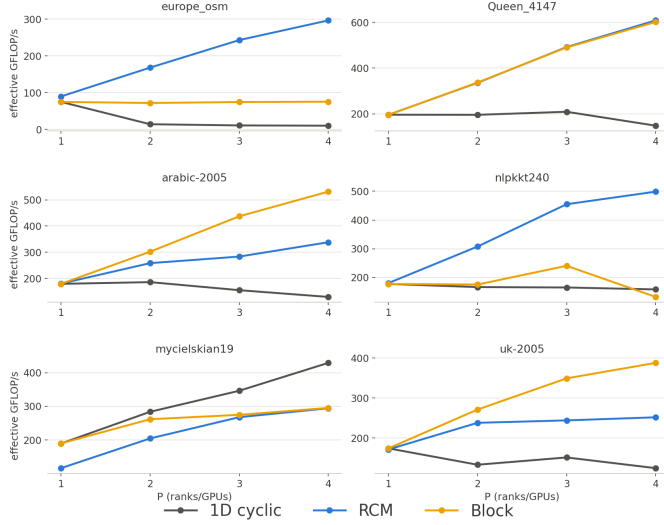


Fig. 4: Aggregate effective GFLOP/s vs. P (cuSPARSE kernel).

changes the *partitioning function*, evaluating RCM and Block against row-cyclic.

The effect is strong and broad (Figures 4–5). On *europe_osm*, cyclic degrades $7.5\times$ from $P=1$ to $P=4$ (2.89 ms $\rightarrow$ 21.64 ms) while RCM improves *monotonically* over the same range (2.41 ms $\rightarrow$ 0.73 ms): a $29.7\times$ margin at $P=4$, the cleanest single result. Overall, RCM turns cyclic’s comm-bound degradation into real improvement on 6 of 10 matrices, and zero-preprocessing Block independently wins on 5/10 (those whose original row numbering already carries usable locality). Communication volume, not kernel throughput, is the mechanism throughout (Figure 5): compute stays flat across strategies, while *europe_osm*’s communication time alone drops from 20.79 ms (cyclic) to 0.11 ms (RCM).

Two counter-cases stand out: *mycielskian19*, already compute-bound under cyclic, gains nothing from RCM ($0.69\times$, pure preprocessing overhead); and *webbase-2001* (1.02B nnz) runs out of memory under both strategies (RCM at $P=4$, Block already at $P=3$).

H. Interconnect Sensitivity

Figure 5 breaks down the bottleneck rank’s compute/communication time at $P=4$ under 1D ghost/NCCL. Every matrix (except the dense, compute-bound *mycielskian19*) is communication-bound, from a narrow

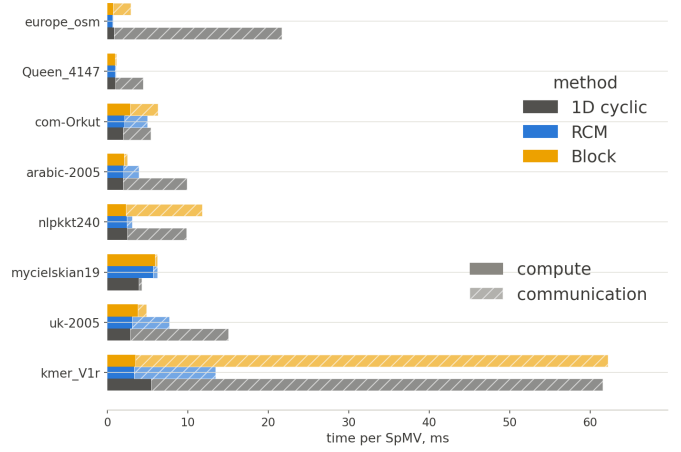


Fig. 5: Compute vs. communication breakdown (cuSPARSE kernel).

1.8:1 ratio on *com-Orkut* to 24.5:1 on *europe_osm* (too sparse to amortize communication against).

Combined with this node’s PCIe-only interconnect (Section III-A), the implementation is unambiguously *interconnect-bound* on every matrix except *mycielskian19*.

V. CONCLUSIONS

This work distributed SpMV across four GPUs with GPU-aware MPI, validating correctness against a serial reference on ten large SuiteSparse matrices. Row-cyclic partitioning with selective ghost exchange is communication-bound on 9 of 10 matrices under this node’s PCIe-only interconnect, and strong scaling degrades with P accordingly, regardless of transport. Swapping MPI point-to-point for NCCL improves the constant factor substantially (median $5.3\times$ at $P=4$) but leaves this direction unchanged on most matrices. Changing the partitioning itself does more: a 2D checkerboard layout capping communication peers at $\sqrt{P}$ wins on 9/10 matrices at $P=4$, and RCM, an alternative to row-cyclic, reverses the degradation into genuine improvement on 6/10 matrices, at the cost of extra preprocessing: partitioning quality, not transport speed, is the primary lever for real multi-GPU strong scaling. The structured-vs-unstructured comparison reinforces this: ghost-exchange saturation, not a matrix’s structured/power-law label, predicts whether selective exchange pays off.

REFERENCES

- [1] J. Gao, B. Liu, W. Ji, and H. Huang, “A systematic literature survey of sparse matrix-vector multiplication,” *arXiv preprint arXiv:2404.06047*, 2024.
- [2] L. Agostinelli, “Sparse matrix-vector multiplication on gpu: A format and kernel strategy comparison,” 2026.
- [3] L. Agostinelli, *Source code and complete results*, <https://github.com/LucaAgostinelli/GPU-d2>, 2026.
- [4] J. Gao, W. Ji, and Y. Wang, “Optimization of large-scale sparse matrix-vector multiplication on multi-gpu systems,” 2024. DOI: 10.1145/3676847.